Akira Astrum

Blog de astrología con intranet de administración y calculadora de compatibilidad

TRABAJO FINAL DE MÁSTER · DEFENSA

Ivan Maldonado

akira-astrum.vercel.app · github.com/nikaivan/Akira_Astrum

PROBLEMA Y OBJETIVO

Un sistema real, con IA bajo control humano

Blog público

Solo lectura: entradas publicadas y categorías. Sin registro de usuarios.

Intranet

Acceso exclusivo del administrador: crear, editar, publicar, despublicar y borrar entradas y categorías, con imágenes.

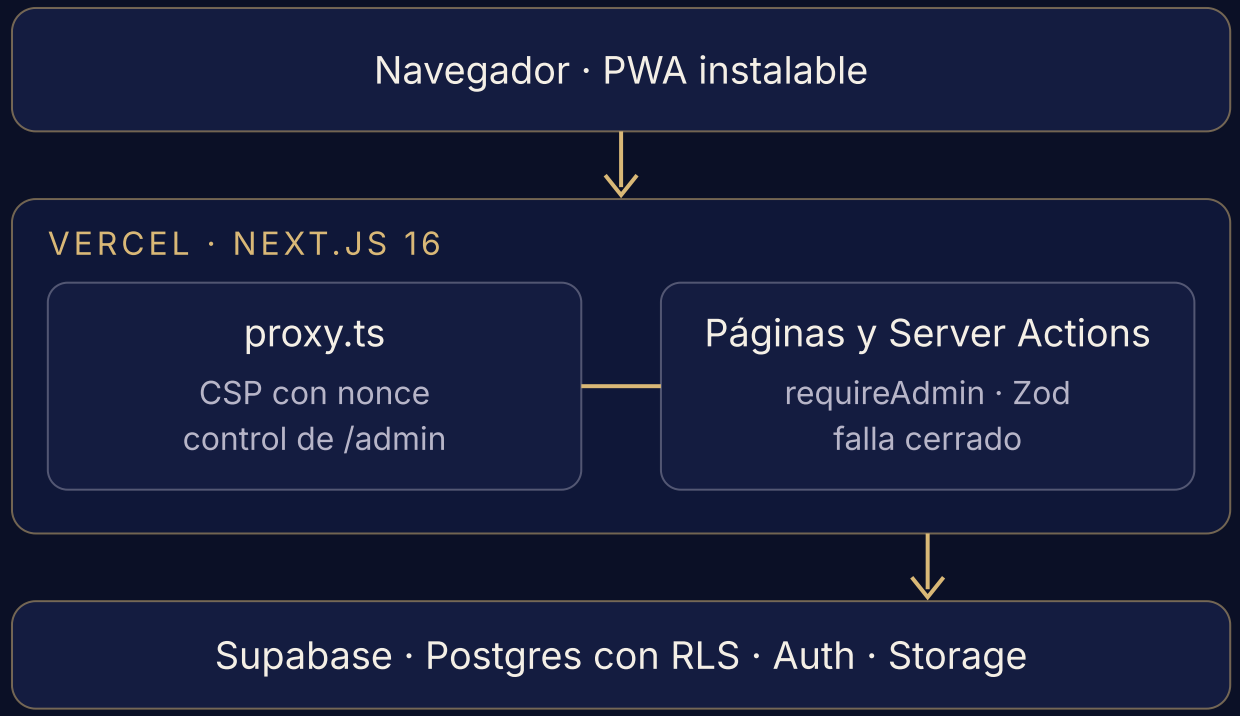
Calculadora

Pública y sin cuentas: compatibilidad entre signos solares. No se guarda ningún dato.

Objetivo. Demostrar que una IA puede implementar un sistema completo si el humano actúa como arquitecto y tester: **TDD estricto**, seguridad según **OWASP Top 10:2025**, instalable como PWA y con coste de 0 €.

ARQUITECTURA

Un monolito Next.js sobre Supabase



- **Monolito** Next.js + Supabase (ADR 0001).
- **Autorización en capas:** proxy, `requireAdmin` y RLS (ADR 0010).
- `service_role` **nunca** en el código de la aplicación.
- 4 migraciones SQL: esquema, acceso, Storage y `cover_alt`.
- Stack: React 19, TypeScript, Zod, Tailwind v4 y shadcn/ui.

METODOLOGÍA

TDD estricto: rojo, verde, refactor

1 · Plan aprobado | 2 · Test en rojo `test:` | 3 · Mínimo en verde `feat:` | 4 · Refactor y ADR

✕ Rojo `4b66f20 test:` la cabecera exigida por ZAP aún no existe

```
// tests/unit/security/cross-origin-policy.test.ts
it("Cross-Origin-Resource-Policy: same-origin", () => {
  expect(headers["cross-origin-resource-policy"]).toBe("same-origin");
});
```

✓ Verde `c61af6d feat:` una línea en la configuración

```
// lib/security/headers.ts
{ key: "Cross-Origin-Resource-Policy", value: "same-origin" },
```

2441
tests unitarios en 102 archivos

13 + 13
archivos de integración y specs e2e (escritorio y Pixel 7)

141
commits y 31 PR fusionadas

SEGURIDAD · OWASP TOP 10:2025

Cada categoría, con su evidencia ejecutable

CAT.	CATEGORÍA	MEDIDAS	EVIDENCIA CLAVE
A01	Control de acceso	✓ 5/5	`supabase/migrations/…_access_control.sql` · `tests/integration/posts-unprivileged.test.ts`
A02	Configuración de seguridad	◖ 8/9	`lib/security/csp.ts` · `tests/unit/security/csp.test.ts`
A03	Cadena de suministro	◖ 4/6	`tests/unit/supply-chain/dependencies.test.ts` · ADR 0021
A04	Fallos criptográficos	✓ 5/5	`.gitignore` · `tests/unit/auth/architecture.test.ts`
A05	Inyección	✓ 7/7	`lib/markdown/sanitize-schema.ts` · `tests/unit/markdown/sanitize.test.ts`
A06	Diseño inseguro	✓ 4/4	`docs/threat-model.md`
A07	Fallos de autenticación	◖ 5/7	`lib/auth/rate-limit.ts` · `tests/unit/auth/rate-limit.test.ts`
A08	Integridad de software y datos	◖ 4/6	`.github/workflows/ci.yml`
A09	Registro y alertas	◖ 2/4	`lib/security/events.ts` · `tests/unit/security/events.test.ts`
A10	Condiciones excepcionales	✓ 4/4	`lib/auth/authorize.ts` · `tests/unit/auth/authorize.test.ts`

57 medidas: 48 hechas · 4 parciales · 5 no hechas — lo no hecho figura como tal en `docs/owasp-checklist.md`

ENDURECIMIENTO

Defensa en profundidad y escaneo ZAP

- **CSP con nonce** por petición y cabeceras estáticas (ADR 0019).
- **Límites de intentos** en el login, la calculadora y las subidas.
- **Eventos de seguridad** sin datos personales y respuestas genéricas (ADR 0020).
- **Cadena de suministro:** versiones exactas, lockfile íntegro, `npm audit` en la CI y Dependabot (ADR 0021).
- **Markdown** con lista blanca; imágenes validadas por contenido (ADR 0013 y 0016).

ZAP baseline 2.17.0

Escaneo pasivo del build de producción, segundo escaneo, 2026-11-04

0 alertas altas

1 media: CSRF, evaluada y aceptada con evidencia

1 baja: COEP, aceptada

La de CORP, del primer escaneo, se corrigió con una cabecera.

5 informativas

CALCULADORA

Compatibilidad entre signos solares

- **12 signos y 66 parejas**; reglas en `docs/astrologia-reglas.md`, validadas por mí como fuente de verdad.
- **Sin persistencia:** una Server Action, validada con Zod, que no guarda ni registra las fechas (ADR 0012).
- **Fechas sin zona horaria**, probadas en cuatro zonas (ADR 0007).
- Límite de 30 consultas cada 10 minutos por instancia.
- **Aviso visible:** convenciones de la tradición astrológica, sin base científica.



Resultado de ejemplo: Aries y Leo

PWA Y RESPONSIVE

Instalable, móvil primero y accesible

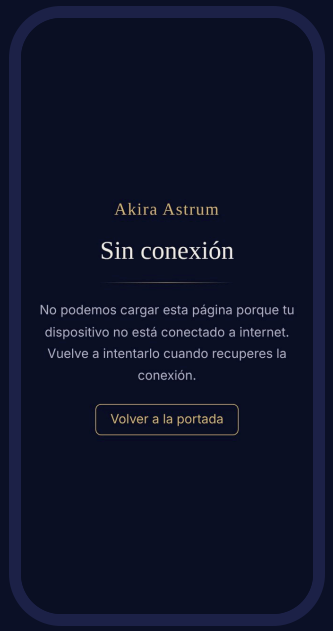
- **Manifiesto** con icono enmascarable y modo *standalone*.
- **Service worker mínimo:** solo sirve `/offline`; nunca guarda contenido ni toca `/admin` ni `/login` (ADR 0022).
- Sin desbordes a 320 px y objetivos táctiles de al menos 44 px.
- Playwright en **escritorio y Pixel 7**, y axe (WCAG 2.1 AA).
- Contraste medido por test con la fórmula de WCAG.



Portada



Calculadora



Sin conexión

DEMO

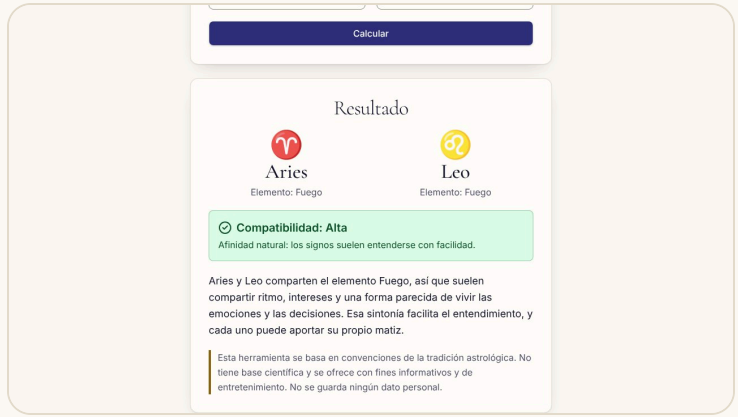
La aplicación en funcionamiento



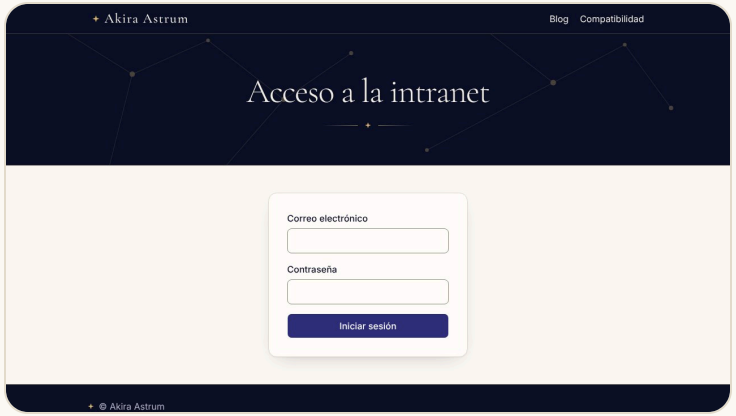
Portada



Blog



Calculadora



Acceso a la intranet



Sin conexión (PWA)

En directo

akira-astrum.vercel.app

Panel: crear, publicar y borrar una entrada con portada.

Capturas del entorno local con datos de ejemplo.

REPARTO DE ROLES

IA implementadora; yo, arquitecto y tester

La IA

- Plan, tests, código mínimo y refactor.
- ADR y entradas del `ai-log`.
- Verificación con `next build` y un Chromium real.

Yo, arquitecto

- Reglas del proceso y aprobación de cada plan, test y commit.
- Decisiones en los puntos ambiguos.
- Aprobación de cada ADR.

Yo, tester

- Integración contra Supabase real y e2e.
- ZAP en mi Mac y la comprobación del despliegue.
- Fallos que las suites no veían.

Errores reales de la IA, registrados en `docs/ai-log.md`:

- El script de ZAP salía **en silencio** con mi formato de `.env`; ninguna prueba lo vio hasta que lo ejecuté yo.
- Agrupó en Dependabot un parche de React con saltos mayores y rompió la CI.
- Repitió fallos ya documentados (`?raw` con CSS, un módulo llamado `register`).

LIMITACIONES

Lo que no está hecho, declarado

- Sin **autenticación multifactor**.
- Sin **alertas automáticas** ni registro de las acciones del admin.
- El despliegue **no consta condicionado** a la CI en verde.
- Limitadores **en memoria por instancia**.
- Acciones de GitHub fijadas por versión, no por hash.
- Listados limitados a **50 entradas**, sin paginación.
- COEP no activada: aviso bajo de ZAP aceptado.
- 2 avisos de dependencias de **solo desarrollo**.
- Caché del service worker entre versiones: sin comprobar por separado.
- El proyecto gratuito de Supabase **se pausa por inactividad**.

Cada punto consta en `docs/owasp-checklist.md` y en el README del repositorio.

LECCIONES Y CONCLUSIONES

Qué me llevo

Lecciones

- Probar con los **datos y formatos reales** del usuario.
- `next build`, integración y e2e en **cada cierre** de bloque.
- Releer los ADR y el registro antes de escribir tests parecidos.
- CI y Dependabot desde las **primeras fases**.
- **Medir el tiempo** desde el principio.

Conclusiones

- Con TDD y revisión humana, la IA entregó una aplicación **desplegada** y probada: 2441 tests unitarios y 23 ADR.
- OWASP: **48 de 57 medidas** con evidencia ejecutable; el resto, declarado.
- El control humano no es un trámite: **detectó errores** que las pruebas de la IA no veían.

Gracias